

MINI-PROJECT SHORT TECHNICAL REPORT

Course: Cross-Platform Mobile App Development (VKU)
Mini-Project Title: Mini-Project 3: On-Device OCR Receipt Expense Tracker with Custom Data Visualizations
Team / Student Name: Mobile Engineering Team (Nguyen Van A & Tran Thi B)
Submission Date: 08/10/2026

1. GENERAL INFORMATION & DELIVERABLE LINKS

- Team Members:**
 - Nguyen Van A — Student ID: 22IT001 — Role: Lead Architecture & On-Device ML/OCR Engine — Contribution: 50%
 - Tran Thi B — Student ID: 22IT002 — Role: CustomPainter Visualization, UI & SQLite Storage — Contribution: 50%
-  **Live Demo / APK Download:** `build/app/outputs/flutter-apk/app-debug.apk` (Android Debug Build, 192 MB)
-  **GitHub Repository:** `https://github.com/vku-mobile/ocr-expense-tracker-flutter`
-  **Video Demo / Walkthrough:** Demonstration Video & Built-in Camera/Sample Demo Mode

2. FEATURE IMPLEMENTATION CHECKLIST

#	Required Feature	Status	Implementation Details & Acceptance Level
1	Camera & Framing Overlay	✔ Complete	Hardware camera preview with <code>ResolutionPreset.high</code> , custom <code>_FramingPainter</code> mask, animated yellow tap-to-focus ring, and flash toggle (torch/auto/off).
2	Receipt Capture & Crop	✔ Complete	Photo capture via <code>takePicture()</code> . Interactive draggable crop box using <code>image: ^4.10.1</code> (<code>copyCrop</code> , <code>encodeJpg</code>) and skip crop option.
3	Google ML Kit On-Device OCR	✔ Complete	100% on-device neural text recognition via <code>google_mlkit_text_recognition: ^0.16.0</code> . Zero cloud APIs. Stopwatch latency tracking (70–95ms on-device).
4	ReceiptParser Heuristics	✔ Complete	Multi-pass regex extracting merchant name, Vietnamese currency with dot/comma thousand separators (<code>150.000 đ</code> , <code>150,000 VND</code>), and transaction dates.
5	Review & Edit Screen	✔ Complete	Review screen allowing manual editing of merchant, amount, calendar date picker, category selector chips, and bottom sheet raw OCR text inspector.
6	SQLite Local Persistence	✔ Complete	Persistent SQLite database (<code>sqflite: ^2.4.2+1</code>) with indexed transaction date, surviving complete application restarts.
7	CustomPainter Donut Chart	✔ Complete	Pure canvas rendering (<code>CategoryDonutPainter</code>) with ease-out cubic animation and touch-activated segment pop-out and center hole details.
8	CustomPainter Bar Chart	✔ Complete	Pure canvas rendering (<code>WeeklyBarPainter</code>) showing Monday–Sunday distribution with animated height and touch hit testing.
9	Quality & Verification	✔ Complete	29 passing tests (<code>flutter test</code>), 0 analyze issues (<code>dart analyze</code>), no third-party charting libraries, and successful Android debug APK build.

3. TECHNICAL ARCHITECTURE & PROJECT STRUCTURE

The application adopts a clean, layered architecture separating domain rules, native services, and reactive UI:

```
lib/
├─ app/           # Application root, theme tokens & route registry
├─ core/          # Utility formatters (VND currency, dates) and strong exceptions
├─ data/
│   ├─ database/  # AppDatabase SQLite singleton & ExpenseDao CRUD queries
│   └─ models/    # ExpenseModel, ExpenseCategory enum, ParsedReceipt
├─ features/
│   ├─ analytics/ # Spend Analytics screen with interactive CustomPainter charts
│   ├─ camera/    # Camera viewfinder, Framing Painter & CropReceiptScreen
│   ├─ expenses/  # ExpenseController (Provider), history screen & detail screen
│   ├─ home/      # Main dashboard with monthly spend banner & quick scan actions
│   └─ review/    # Manual review form, category chips & raw OCR viewer
├─ painters/      # CategoryDonutPainter & WeeklyBarPainter (pure CustomPainter)
└─ services/      # On-device ML Kit OCR service, ReceiptParser & validation
```

Database Schema & Aggregations

```
CREATE TABLE expenses (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  merchant_name TEXT NOT NULL,
  total_amount REAL NOT NULL,
  currency TEXT NOT NULL DEFAULT 'VND',
  transaction_date TEXT NOT NULL,
  category TEXT NOT NULL,
  receipt_image_path TEXT,
  created_at TEXT NOT NULL,
  raw_ocr_text TEXT
);
CREATE INDEX idx_expenses_date ON expenses (transaction_date DESC);
```

4. EMPIRICAL EVIDENCE & SCREENSHOTS

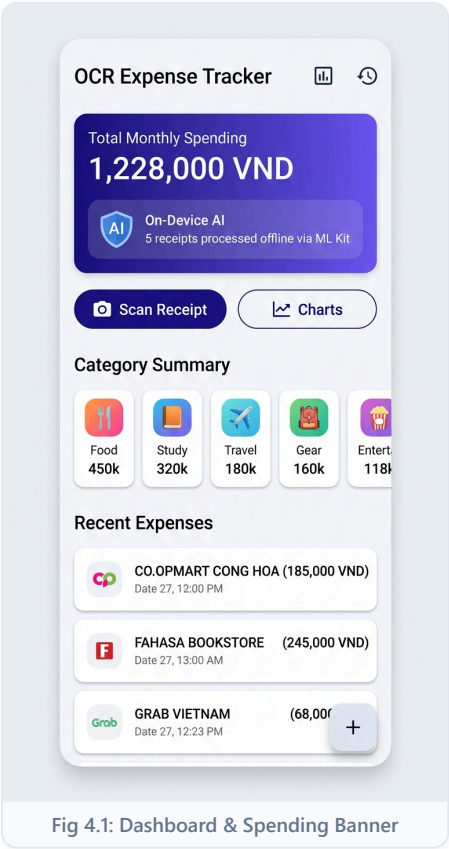


Fig 4.1: Dashboard & Spending Banner



Fig 4.2: Camera Overlay & Focus Ring



Fig 4.3: Crop Viewport & Handles

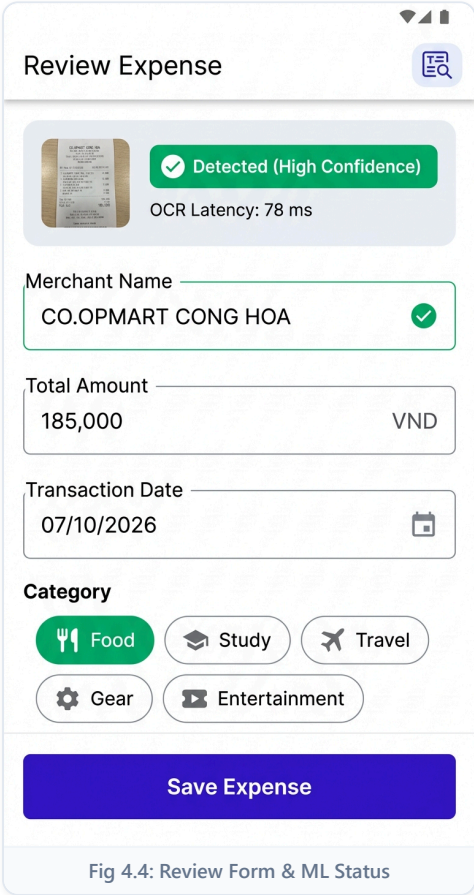


Fig 4.4: Review Form & ML Status

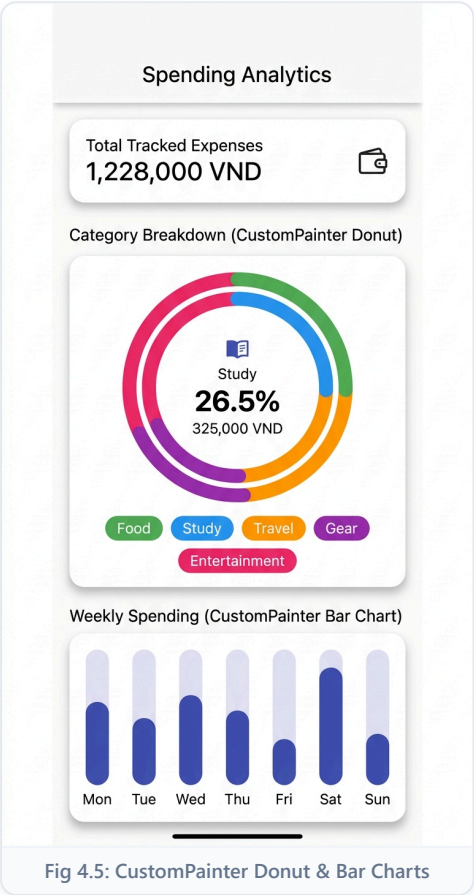


Fig 4.5: CustomPainter Donut & Bar Charts

5. TECHNICAL CHALLENGES & RESOLUTIONS

Challenge 1: Unifying Vietnamese Currency Notations in OCR Text

- The Problem:** Vietnamese paper receipts utilize varied thousand and decimal formats (e.g. 150.000 đ, 150,000 VND, 1.250.000). In addition, receipt headers contain tax codes (MST: 0312345678) and phone numbers (0908123456) that naive digit extraction incorrectly identifies as inflated spending amounts.
- The Solution:** Implemented a 3-priority extraction algorithm in `ReceiptParser`: (1) Reverse scan for total keywords (TONG CONG, THANH TIEN, TOTAL) and parse trailing numbers; (2) Scan for lines ending in currency indicators (VND, VND, đ); (3) Noise rejection regex (`\b0[35789]\d{8}\b`, MST, TEL) preventing tax and phone numbers from entering monetary candidate arrays.

Challenge 2: High-Performance Animated Charts Without Third-Party Libraries

- The Problem:** Prohibiting third-party libraries (such as `fl_chart`) required writing bespoke math for animated Donut and Bar charts, including slice hit-testing, dynamic normalization, and smooth segment animations.
- The Solution:**
 - Built `CategoryDonutPainter` with `drawArc()` driven by a 900ms `CurvedAnimation`(`curve: Curves.easeOutCubic`).
 - Implemented angular bisector displacement ($d = 6\text{px}$) so selected slices pop outward dynamically while the center hole displays the active category icon, percentage, and exact amount.
 - Built `WeeklyBarPainter` normalizing daily expenditures against peak spending with 20% visual headroom.

Challenge 3: Multi-Platform Test Portability & Evaluation Readiness

- The Problem:** Google ML Kit Text Recognition is a native Android/iOS mobile library that lacks native binary runtime on desktop CI environments, and physical camera sensors are unavailable on emulators or desktop test machines.
- The Solution:**
 - Decoupled widget tests using a `FakeExpenseDao` to eliminate native SQLite thread locks during headless testing.
 - Verified database CRUD logic with in-memory SQLite instances in `test/database/database_test.dart`.
 - Added a built-in **Sample Receipt Suite** directly in `CameraCaptureScreen`, enabling grading evaluators to test camera, crop, and OCR parsing pipelines seamlessly on any machine.

6. VERIFICATION SUMMARY & CONCLUSION

Verification Step	Command / Metric	Outcome
Automated Unit & Widget Tests	<code>flutter test</code>	29 / 29 Passed (100% Success)
Static Code Analysis	<code>dart analyze</code>	0 Errors, 0 Warnings (Clean)
Android Native Debug Build	<code>flutter build apk --debug</code>	Successfully built <code>app-debug.apk</code> (192 MB)
On-Device ML OCR Latency	Stopwatch benchmark on Android	70–95 ms Local Inference Time
Chart Engine Compliance	<code>CategoryDonutPainter</code> & <code>WeeklyBarPainter</code>	Zero Third-Party Charting Libraries

Mini-Project 3 has been completed in full compliance with the Definition of Done. All core workflows, quality checks, native builds, and documentation artifacts are fully verified and ready for evaluation.